CUDA Kernel Engineering

Lecture notes from the CudaKernels project — seven kernels, measured and explained

Hardware: NVIDIA GTX 1060 Max-Q (Pascal, sm_61)

Kernels: matmul · dot product · transpose · GEMV · scan · 1D convolution · 2D convolution

Compiled July 2026 — all numbers measured on this machine, all kernels written by hand

Contents

1 The card you are programming

Every number in these notes was measured on one specific GPU, and its limits shape every decision, so know them cold:

Property	Value	Why it matters
GPU	GTX 1060 Max-Q (GP106, Pascal)	compute capability 6.1 (`sm_61`)
SMs (streaming multiprocessors)	10	the independent "cores" that run blocks
fp32 peak	~3.9 TFLOP/s	ceiling for arithmetic
DRAM peak bandwidth	~192 GB/s	ceiling for streaming data
L2 cache (chip-wide)	1.5 MB	shared by all SMs
Shared memory per SM	96 KB	the pool blocks lease slices of
Max threads per SM	2048	occupancy = resident warps / 64
Warp size	32	the true unit of execution

It is a laptop part. Clocks drift with temperature, so absolute timings swing ±10% run-to-run (and up to ~50% for barrier-heavy kernels across cold/hot states). Two rules follow: benchmark with the *median* of several samples, and when comparing two versions, trust the **ratio measured in the same run**, never absolute numbers from different days.

2 The execution model: grids, blocks, warps

A kernel launch creates a **grid** of **blocks**; each block is a group of up to 1024 **threads**. You choose both shapes at launch: `kernel<<<grid, block>>>(...)`. The hardware assigns whole blocks to SMs, where they stay until they finish.

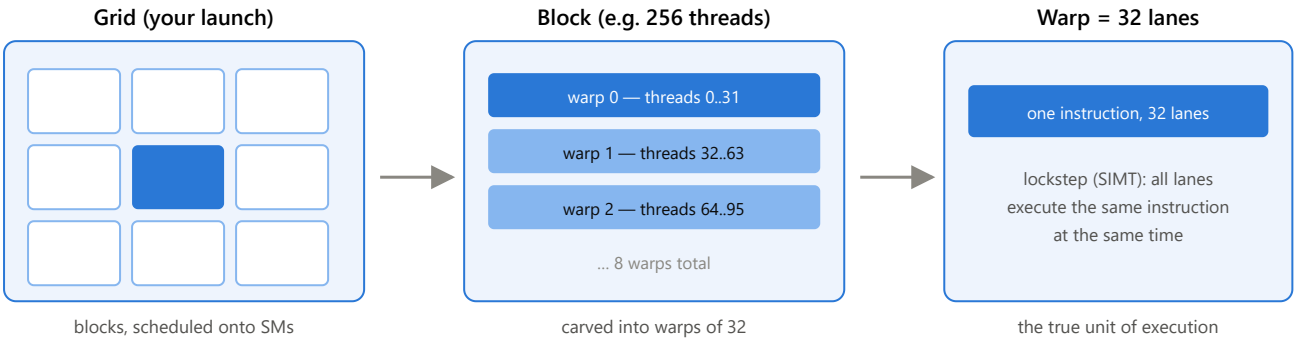


Figure 2.1 — The three levels. You program threads, but the hardware executes *warps*: 32 threads marching in lockstep. Almost every performance effect in these notes (coalescing, divergence, bank conflicts, shuffles) is a warp-level effect.

2.1 Which threads share a warp — it depends on block shape

Threads are packed into warps by their linear index `tid = threadIdx.z·(Dy·Dx) + threadIdx.y·Dx + threadIdx.x` — x varies fastest. So in a 256-thread 1D block, warp 0 is threads 0–31. In a 16×16 block, a warp is **two adjacent rows of 16**. In a 32×8 block, a warp is **exactly one row of 32**. This seemingly cosmetic detail decided the 2D convolution's bank-conflict story (chapter 11).

2.2 Lockstep and its consequences

- **Branching:** when lanes of one warp disagree on an `if`, the hardware runs each side with the other lanes switched off (divergence) — or the compiler avoids branching entirely by **predication**: every lane executes the instruction, a per-lane flag decides whether the result is kept. Either way the cost of a one-sided `if` is at most the body once; the real bill is usually the *condition evaluation itself*, paid by every warp (chapter 11).
- **Memory:** a warp's 32 loads are issued together, and the hardware merges them into as few memory transactions as possible — this is *coalescing* (chapter 3).
- **Cooperation:** lanes can exchange registers directly with shuffle instructions — no memory involved (chapter 6).

2.3 Latency hiding: why the GPU wants thousands of threads

A single load from DRAM takes hundreds of cycles. A CPU hides that with caches and speculation; a GPU hides it with **other warps**: when warp A stalls on a load, the SM instantly switches to warp B. The more resident warps an SM has (its **occupancy**, out of a maximum of 64), the more stalls it can paper over. Anything that reduces resident warps — big shared-memory allocations, high register counts — reduces the SM's supply of "other work" (chapters 8 and 11 both hinge on this).

3 The memory hierarchy

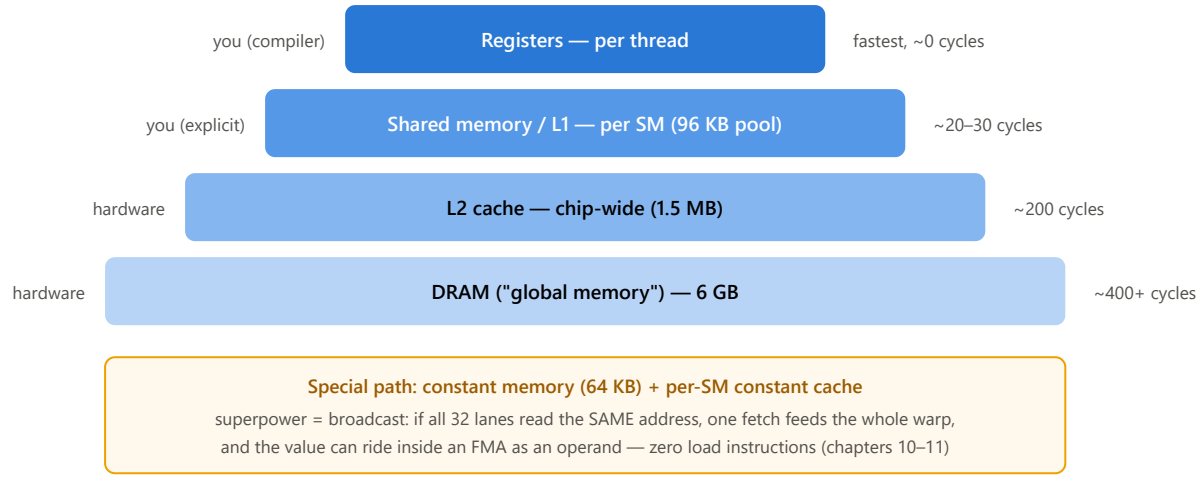


Figure 3.1 — The hierarchy with this card's numbers. Left labels say who manages each level. The two levels *you* control — registers and shared memory — are where every optimization in Part II lives.

3.1 Coalescing: the first rule of global memory

DRAM is read in 32-byte sectors grouped into 128-byte cache lines. When a warp's 32 lanes load 4-byte floats from **32 consecutive addresses**, the whole warp is served by one 128-byte line: perfect. When lanes load addresses a full row apart (stride N), the warp needs 32 separate transactions — a 32× traffic amplification for the same useful data. GEMV (chapter 8) measures this exact difference: 43 vs 151 GB/s.

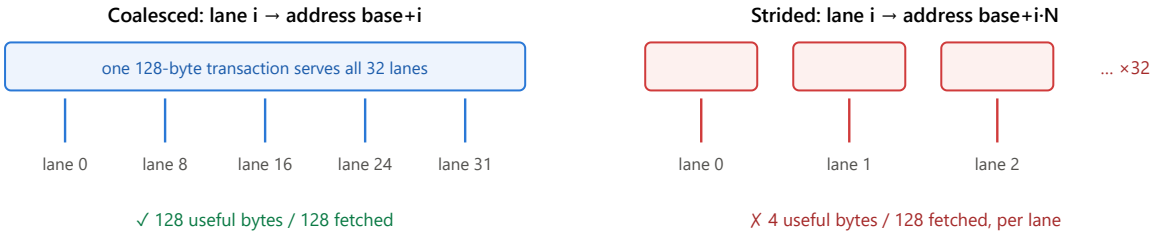


Figure 3.2 — Coalescing. Same 32 floats of useful data; up to 32× difference in memory traffic. Design the *thread-to-data mapping* so that consecutive lanes touch consecutive addresses.

3.2 Shared memory: a leased scratchpad, not a cache

Each SM owns 96 KB of shared memory. When a block launches, it gets an **exclusive slice** sized to its request, held until the block dies; inside the kernel, `__shared__` arrays live in that slice. Three consequences, each learned the hard way:

- **It is a purchase, not a free win.** Every KB you request is a bid against occupancy: ask for 16 KB/block and only 6 blocks fit per SM (GEMV, chapter 8 — a measured regression).
- **It starts as garbage.** The previous tenant's data is still there. Every slot your compute loop reads must be written first (convolution halos, chapters 10–11).
- **It is private to the block.** No block can read another's slice — cross-block cooperation must round-trip through global memory between kernel launches (scan, chapter 9).

3.3 Banks: how shared memory is wired

Shared memory is divided into **32 banks**; 4-byte word *i* lives in bank *i* mod 32. A warp reads at full speed only when its 32 lanes hit 32 *different* banks (reading the identical word is also fine — that's a broadcast). Two lanes on the same bank at different addresses = the access runs twice (**2-way conflict**); a whole warp on one bank runs 32× slower. Transpose (chapter 7), scan (chapter 9), and conv2d (chapter 11) each hit a different variant of this — and needed three different fixes.

3.4 A Pascal-consumer quirk that shaped chapter 11

> **SM_61 QUIRK**
> On this card, an *ordinary* global load never uses the per-SM L1 cache — it always goes to L2. The fast L1 path is reserved for the **read-only load** (`LDG.CI`), which the compiler may only emit when it can prove no thread writes that memory during the kernel. Plain `const` is not proof (the same memory could be written through another pointer). Marking pointers `const T* __restrict__` provides the proof. This single annotation was worth 1.4× on conv2d.

4 Measuring honestly: rooflines and ratios

4.1 The roofline: which ceiling are you under?

Every kernel does some arithmetic (FLOPs) and moves some bytes. Its **arithmetic intensity** (AI) is the ratio: useful FLOPs per byte of *ideal* traffic. The machine imposes two ceilings — 192 GB/s of memory bandwidth and 3900 GFLOP/s of arithmetic — and which one binds depends on AI. The crossover ("ridge") for this card is 3900/192 ≈ **20.3 FLOP/byte**.

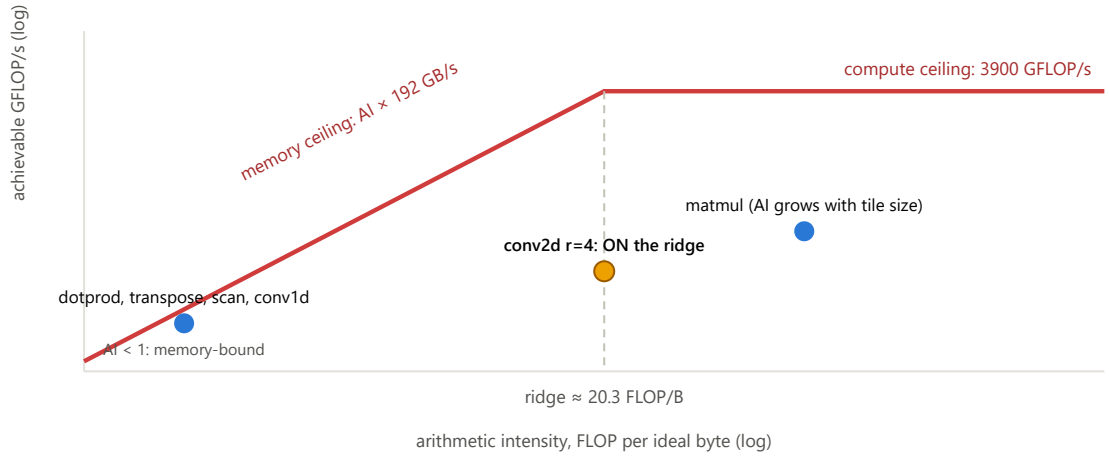


Figure 4.1 — The roofline for this card (schematic). Left of the ridge, only GB/s matters; right of it, only GFLOP/s. Conv2d at r=4 has AI = 162 FLOPs / 8 bytes ≈ 20.3 — exactly at the ridge, so both numbers are reported and whichever is further from its peak points to the slack.

4.2 Effective bandwidth: count ideal traffic, not actual

All GB/s figures in these notes are **effective**: (bytes the kernel *logically must* move, e.g. read input once + write output once) ÷ time. Redundant traffic — halo re-reads, mask fetches, uncoalesced amplification — deliberately does not count as useful. That makes effective bandwidth a score against the ideal: reaching ~110–140 GB/s (what this card actually sustains for streaming) means the kernel wastes nothing.

4.3 The measurement discipline

- **Median-of-samples timing** (a warm-up, then the median of several timed runs).
- **Same-run A/B.** Variants are compared inside one program execution — thermal state cancels out.
- **Correctness gates perf.** Every kernel is checked against a CPU reference first; outputs are poisoned with NaN bytes beforehand so a kernel that writes nothing fails loudly.
- **Tests are hardened against lucky symmetry** — e.g. the conv2d mask is made asymmetric so a transposed index can't silently pass.
- **When profilers fail, improvise:** hardware counters are blocked on this Windows setup, so the diagnostics became controlled sweeps (chapter 10) and instruction-census reads of the compiled assembly via `cuobjdump -sass` (chapter 11).

5 Matrix multiplication — reuse and tiling

C = A·B, 1024^3-class sizes. Measured: naive ~179–191 → shared-tiled ~430 → register-tiled ~1030–1044 GFLOP/s (26% of fp32 peak).

5.1 The problem with naive

One thread per output element: thread (row, col) walks row of A and column of B, doing 2N FLOPs but issuing 2N global loads. Every element of A is re-fetched by a whole row of threads, every element of B by a whole column — the data has enormous **reuse**, and naive throws it all at the caches. AI per thread ≈ 0.25 FLOP/byte: deep in memory-bound territory despite matmul being the textbook compute-bound problem.

5.2 Shared-memory tiling: buy reuse explicitly

Partition C into 16×16 tiles, one block each. The block marches along the shared dimension in 16-wide steps; at each step it cooperatively loads one 16×16 tile of A and one of B into shared memory (each thread loads one element of each — coalesced), barriers, then every thread does 16 FMAs out of the tiles, barriers again, and moves on:

```
for (int t = 0; t < numTiles; t++) {
    Ads[ty][tx] = /* guarded load of A tile element */;
    Bds[ty][tx] = /* guarded load of B tile element */;
    __syncthreads();                        // tiles complete before ANY thread reads
    for (int k = 0; k < TILE_WIDTH; k++)
        value += Ads[ty][k] * Bds[k][tx];
    __syncthreads();                        // everyone done reading before next overwrite
}
```

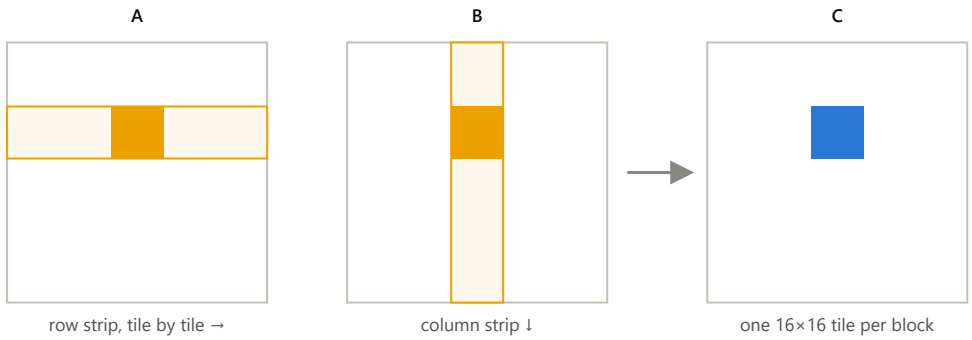


Figure 5.1 — Tiling. Each orange 16×16 tile is loaded from global memory *once* (256 loads) and then read 16 times by each of 256 threads (4096 reads) — a 16× reduction in global traffic. Bigger tile ⇒ more reuse ⇒ higher arithmetic intensity.

5.3 Register tiling: reuse *within* a thread

Shared memory is fast but still ~20× slower than a register. The 2×2 register-tiled kernel makes each thread compute **four** outputs (c00, c01, c10, c11). In the inner loop it reads two values from the A tile and two from the B tile — 4 shared reads — and performs 4 FMAs: a 2× better FMA-per-shared-read ratio than the plain tiled kernel, with the accumulators living in registers.

```
for (int k = 0; k < TILE_WIDTH; k++) {
    c00 += Ads[ty][k]            * Bds[k][tx];
    c01 += Ads[ty][k]            * Bds[k][tx + TILE_WIDTH];
    c10 += Ads[ty + TILE_WIDTH][k] * Bds[k][tx];
    c11 += Ads[ty + TILE_WIDTH][k] * Bds[k][tx + TILE_WIDTH];
}
```

LESSONS

(1) Find the reuse in the math, then choose which level of the hierarchy captures it: caches (naive — bad), shared memory (tiled — 2.4×), registers (register-tiled — another 2.4×). (2) The double-barrier pattern — load, sync, compute, sync — is the canonical shape of every tiled kernel in this repo. (3) At 26% of fp32 peak there is known headroom (double-buffering: prefetch tile t+1 while computing on tile t) — a future session.

6 Dot product — reductions and atomics

N = 16.7M floats. Measured: ~125–138 GB/s effective (~70% of spec peak, which is the practical streaming ceiling). Memory-bound by design.

6.1 Grid-stride loading

Launch a bounded grid (≤1024 blocks) and let each thread accumulate many elements:

```
for (int i = idx; i < N; i += gridDim.x * blockDim.x)
    sum += A[i] * B[i];
```

Consecutive lanes read consecutive addresses at every step (coalesced), any N is handled without grid-size gymnastics, and each thread arrives at the reduction holding one partial sum.

6.2 The three-stage reduction

Summing one value per thread across the whole block:

1. **Warp level — shuffles.** `__shfl_down_sync(mask, val, offset)` hands lane *i* the register value of lane *i+offset*, no memory involved. Halving the offset 16→8→4→2→1 folds 32 partials into lane 0 in five steps.
2. **Block level.** Each warp's lane 0 drops its result into a tiny shared array (`warpSums[32]`); after a barrier, the first warp shuffle-reduces those.
3. **Grid level — one atomic.** Thread 0 of each block does `atomicAdd(C, sum)`. ≤1024 atomics for 16.7M inputs: contention is irrelevant. (The launcher must zero the accumulator first — a contract worth remembering.)

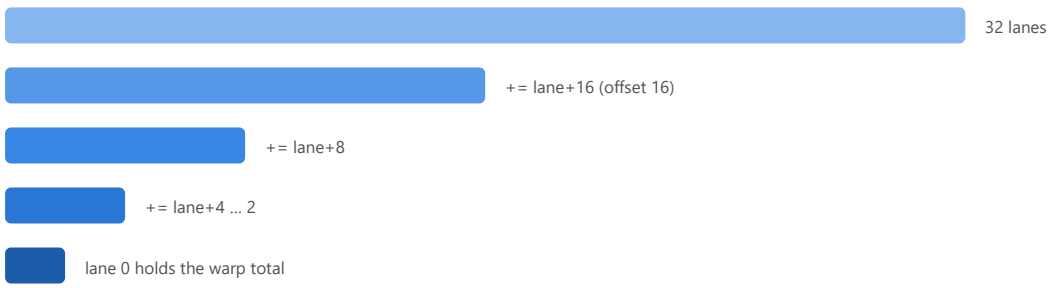


Figure 6.1 — Warp shuffle reduction: $\log_2 32 = 5$ register-to-register steps, no shared memory, no barrier. This helper was written once and reused verbatim in GEMV.

LESSONS — INCLUDING TWO NEGATIVE RESULTS

(1) The kernel is memory-bound: the entire reduction (shuffles + one barrier + one atomic) is invisible next to streaming 128 MB. (2) `float4` vectorized loads **regressed ~5%** — coalesced scalar loads already saturate the transaction width on Pascal, and wider loads reduced loads-in-flight. (3) 4× unrolling was **neutral**. Optimizations that "obviously help" often don't — measure, keep the simple version when it wins.

7 Transpose — coalescing and bank conflicts

2048×4096. Measured: naive ~56–57 → tiled ~82 → padded ~139 GB/s (~72% of peak).

7.1 The structural problem

B[j][i] = A[i][j]. If reads of A are coalesced (consecutive columns), then writes to B are column-order — stride-*rows* apart, uncoalesced. One side must suffer… unless you change the data's *orientation* midway.

7.2 Staging the swap through shared memory

The block loads a 32×32 tile row-wise (coalesced reads into `tile[ty][tx]`), barriers, then writes it out *reading the tile transposed* (`tile[tx][ty]`) with swapped block coordinates — so the global *writes* are also row-order, coalesced. The transpose happens inside shared memory, where "orientation" costs nothing… almost.

7.3 The bank conflict, and the +1 pad

Reading `tile[tx][ty]` means the warp's 32 lanes read down a *column*: addresses 0, 32, 64, … — all congruent mod 32, **all in one bank**. A 32-way conflict: the read serializes 32×, and tiled only reached 82 GB/s. Declaring `tile[32][33]` makes the row stride 33, coprime with 32 — a column's successive elements now land in banks 0, 1, 2, …: conflict-free. That one character (the +1) took the kernel from 82 to 139 GB/s.

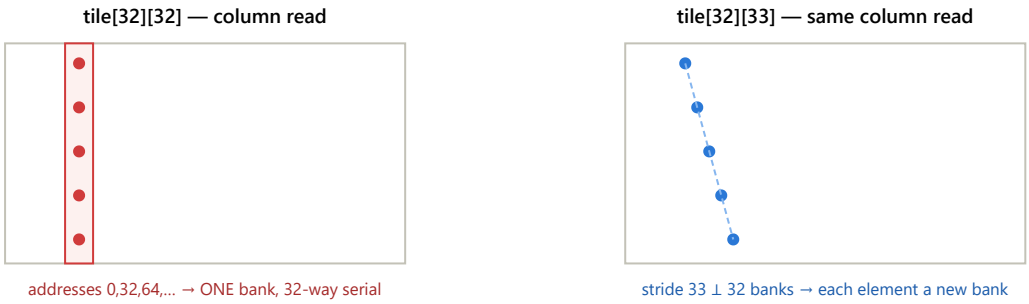


Figure 7.1 — The padding trick. With row stride 32, a column lives entirely in one bank (left). Stride 33 is coprime with the 32 banks, so the column "drifts" diagonally across all of them (right). Cost: 32 wasted words per tile. Gain: 1.7×.

LESSONS

(1) When a kernel's two sides want incompatible access orders, stage through shared memory and give *each* side its preferred order. (2) Bank conflicts are invisible in code and enormous in effect — reason about *addresses mod 32* whenever a warp's shared accesses are strided. (3) The coprime-stride pad recurs: `[TILE][TILE+1]` here, `i + (i>>5)` in scan.

8 GEMV — access patterns, and a negative result

y = A·x, 4096×4096. Measured: naive ~31–43 → warp-per-row ~151 GB/s (~79% of peak) → warp-per-row + shared-x ~120–130 GB/s (a deliberate, kept regression).

8.1 Thread-per-row vs warp-per-row

The naive mapping gives each *thread* a row. Look at it from the warp's perspective: at step i, lane 0 reads A[0][i], lane 1 reads A[1][i] — addresses a full row (16 KB) apart. Stride-N, uncoalesced, ~32× traffic amplification (Figure 3.2 in the flesh). The fix is to flip the mapping: give each **warp** a row. Lane *l* strides the columns by 32 — consecutive lanes, consecutive addresses, one transaction — and a warp shuffle collapses the 32 partials:

```
for (int i = lane; i < N; i += 32)
    sum += A[wid * N + i] * x[i];
sum = warpReduceSum(sum);
if (lane == 0) y[wid] = sum;
```

43 → 151 GB/s. Nothing about the arithmetic changed — only *who reads what*.

8.2 The shared-x experiment: a negative result worth keeping

Every row reads the same x (16 KB). Hypothesis: stage x in shared memory once per block. Measured: **15–20% slower**. Two stacked reasons:

1. **L2 already had it.** x is re-read by all 4096 rows, so after first touch it lives in the 1.5 MB chip-wide L2. The staging loop's reads hit L2 anyway — zero DRAM traffic saved — while adding 4096 loads, 4096 shared stores, and a barrier to every block.
2. **Occupancy collapse.** 16 KB/block means [96/16] = 6 blocks/SM instead of 16: resident warps fell 64 → 24 (100% → 37.5%), leaving the SM too few warps to hide A's load latency.

> **THE RULE THIS BOUGHT**
> Shared memory is a *purchase*: you pay in occupancy and staging work, and the return is (reuse count) × (per-access saving vs where the data would otherwise live). If the data already lives in L2, the saving is nearly zero and the purchase is a loss. Matmul tiles and convolution halos pay off; a small read-only vector does not. Before staging anything, ask: **where would the caches put this anyway?**

9 Scan — dependencies, trees, and barriers

*Inclusive prefix sum, N = 1M. Measured: 3-pass Hillis-Steele ~0.27 ms → Blelloch pass 1 ~0.27 ms (1.07×) → Blelloch + padded indexing ~0.19 ms (**1.45–1.7×**, the number to trust is the same-run ratio).*

9.1 Why scan is different

Everything before this chapter was embarrassingly parallel: out[i] depended only on in[i] and neighbors. Scan's out[i] = x[0]+…+x[i] depends on *everything before it*. Parallelizing it means restructuring the arithmetic into a tree — and coordinating across blocks, which have no way to talk to each other during a launch.

9.2 Two single-block algorithms

Hillis-Steele (inclusive): for d = 1, 2, 4, …: every thread does `s[tid] += s[tid-d]` (register-buffered with two barriers per level to avoid the in-place race). $\log_2 N$ levels, but N adds per level: **O(N log N) total work**, dead simple.

Blelloch (exclusive, work-efficient): two elements per thread. An **up-sweep** builds a sum tree in place (strides doubling 1, 2, 4, …), then the root is cleared to 0 and a **down-sweep** walks back down (strides halving), each node doing swap-and-add:

```
// up-sweep level (stride = offset):        // down-sweep level:
int l = offset*(2*tid + 1) - 1;             float t = s[l];
int r = offset*(2*tid + 2) - 1;             s[l] = s[r];
s[r] += s[l];                               s[r] += t;
```

Total work O(N) — asymptotically better. Hold that thought.

9.3 Multi-block: the 3-pass structure

Blocks can't see each other's shared memory, and the only grid-wide synchronization is a kernel boundary. So: **pass 1** — each block scans its 1024-chunk and writes its chunk total; **pass 2** — one block exclusive-scans the totals into per-chunk offsets; **pass 3** — every element adds its chunk's offset. Three launches, global memory as the mailbox.

9.4 The two optimizations, and the upset

Profiling showed pass 1 taking ~80% of runtime at ~26 GB/s while pass 3 — same data volume, no barriers — ran at ~117 GB/s. Pass 1 was **barrier-bound**, not memory-bound. Two candidate fixes:

Fix	Theory says	Wall clock says
Blelloch (O(N) work instead of O(N log N))	huge	**1.07×**
Pad shared indices `i → i + (i>>5)`	footnote	**1.7×**

Why: both algorithms hit ~20 barriers per block, and barriers dominated — so cutting *work* barely moved anything. But Blelloch's power-of-two strides are bank-conflict machines: at stride 32, a whole warp lands in one bank (32-way serial). One pad word every 32 real words (the 1-D cousin of the transpose +1 pad) removed the conflicts and delivered 10× more speedup than the asymptotic improvement.

LESSONS

(1) Kernel boundaries are the only cheap grid-wide sync; design multi-block algorithms as passes. (2) Big-O counts operations; the GPU bills for *barriers and bank conflicts*. Profile before trusting asymptotics. (3) Same-run A/B ratios are the only trustworthy currency on thermally drifting hardware. Known headroom: stride-2 loads and pass-3 fusion (decoupled lookback).

10 1D convolution — constant memory and halos

N = 16.7M, radius r = 4 (9 taps), zero-padded. Measured: naive ~29 → const-mask ~72–82 → tiled ~77–87 → tiled+unrolled ~104–112 GB/s — which IS the streaming floor: done in the strongest sense.

10.1 The stencil pattern

out[i] = $\Sigma_{j=-r..r}$ in[i+j]·mask[j+r]. Every input is reused by 2r+1 = 9 outputs; every thread reads the same 9-float mask. Two reuse structures, two tools.

10.2 Constant memory: the broadcast path (2.8×)

The mask index `j+r` depends on the loop counter, not on threadIdx — all 32 lanes read the **same address** every iteration. That is exactly what `__constant__` memory's per-SM cache is built for: one fetch broadcasts to the whole warp, and the compiler folds the value into the FMA as a direct operand — no load instruction at all. Moving the mask from a plain pointer to `c_mask1d` tripled throughput, not because 36 bytes were far away (they sat in L2 either way) but because it deleted 9 of 18 load instructions and 9 L2 round-trips per output. *Anti-pattern for contrast: indexing `__constant__` by threadIdx makes lane addresses diverge and the reads serialize.*

10.3 Tiling with a halo

A block computing outputs [B, B+256) needs inputs [B−4, B+256+4): its range plus **r extra on each side — the halo**, shared with neighbor blocks. The tile (256+2r floats) is loaded cooperatively with a stride loop, out-of-range slots getting 0.0f — which bakes the boundary handling into the data and leaves the compute loop with *no bounds checks*:

```
for (int s = threadIdx.x; s < tileSize; s += blockDim.x)
    tile[s] = (inRange(g)) ? in[g] : 0.0f;      // g = blockStart - r + s
__syncthreads();
for (int j = -r; j <= r; j++)
    sum += c_mask1d[j + r] * tile[r + threadIdx.x + j];
```

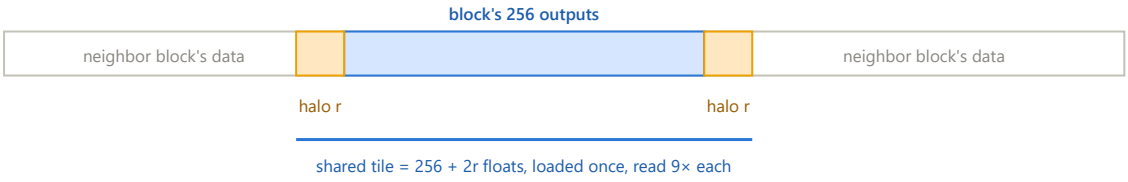


Figure 10.1 — The halo. Orange cells are inputs the block needs but whose outputs belong to neighbors; adjacent blocks each load their own copy. Zero padding is written into the tile at load time, so boundary logic vanishes from the hot loop.

10.4 The radius sweep: time = floor + taps × price

With hardware counters unavailable, a controlled experiment replaced the profiler: run every variant at r = 1, 2, 4, 8 and regress time against tap count. Every variant shares one intercept — the **~1.2 ms streaming floor** (128 MB at the card's real ~110 GB/s) — and differs only in the **per-tap price, set by where that tap's operands live**:

Variant	Per-tap price	Why
naive	~0.45 ms/tap	two L2 trips (input + mask)
const-mask	~0.10 ms/tap	one L2 trip (mask is free broadcast)
tiled / unrolled	~0.04 ms/tap	shared memory

The table also exposes the **crossover**: tiling has a fixed cost (tile load + barrier), so const wins for r ≤ 2 and tiled only pays from r ≈ 4 — shared memory purchased with too little reuse is the GEMV lesson again, in miniature.

10.5 Compile-time radius: `template <int R>`

A runtime-r loop can't unroll: the compiler must keep the counter, compare, and branch, and can't batch the loads. Making the radius a template parameter stamps out one kernel per radius (dispatched by a `switch`, generic fallback for odd radii); `#pragma unroll` then flattens the 9 taps into straight-line code with all loads in flight together — **1.4×** over generic tiled, landing on the streaming floor: time flat from 3 to 9 taps.

> **LESSONS**
> (1) Warp-uniform reads belong in constant memory; the win is broadcast + operand folding, not proximity. (2) The halo pattern: over-fetch by r per side, zero-fill at load, compute check-free. (3) A parameter sweep can decompose runtime into physically meaningful pieces (floor + per-tap price) — a poor man's profiler that's sometimes better than the real one. (4) Values known at compile time are optimization fuel; templates are how you feed them to the compiler.

11 2D convolution — the ladder of walls

*4096×4096 image, 9×9 mask (r = 4, 81 taps), zero-padded. Measured: const-mask 15.0 → +__restrict__ 10.6 → shared-tiled (32×8) 8.4 → template-unrolled **3.6–3.9 ms** (up to ~37 GB/s effective / ~750 GFLOP/s). AI ≈ 20 FLOP/B — exactly the roofline ridge, so both scoreboard numbers matter.*

This chapter is the capstone: four optimizations, and the defining feature of the session was that **each wall was invisible until the previous one fell**. Three predictions missed before one landed — and each miss identified a wall nobody had priced in.

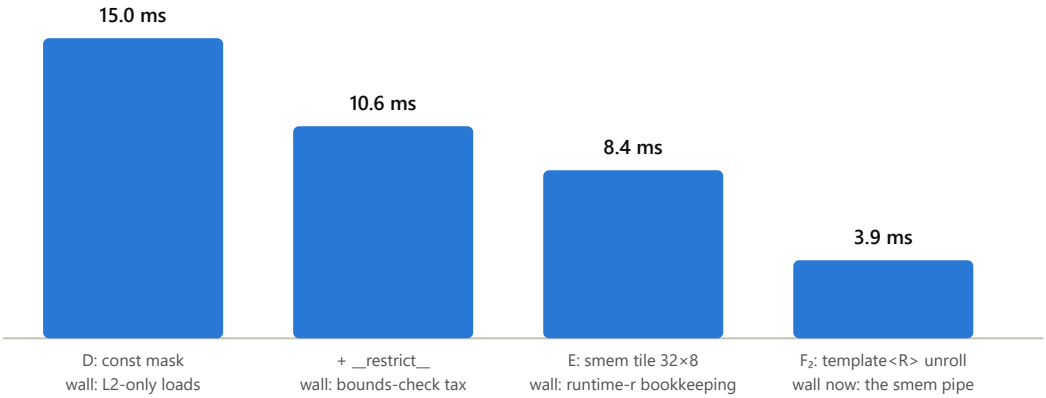


Figure 11.1 — The ladder. Under each bar: the wall that *capped* it, i.e. what the next step had to remove. 3.9× total; the final kernel spends ~75% of its time feeding the shared-memory pipe, which is the honest definition of "the tile is doing its job."

11.1 What changes in 2D

Kernel D is 1D-const in two dimensions: a 2D grid, one thread per pixel, two nested tap loops, per-component bounds checks, mask from constant memory. The geometry, though, changes the memory story:

- **Reuse explodes:** each input pixel feeds $(2r+1)^2$ = 81 outputs, not 9.
- **A warp's row-taps are nearly free:** its 9 horizontal taps span ~2 cache lines — same-line rereads.
- **Its column-taps are not:** vertically adjacent pixels are W·4 = 16 KB apart — 9 distinct lines, and the reuse partners are different warps in different blocks, scheduled whenever.

11.2 Step 1 — `__restrict__` and the read-only path (15.0 → 10.6 ms)

Measured D was 3× worse than the pessimistic prediction. Diagnosis: the sm_61 quirk of §3.4 — plain loads all went to L2. Estimated L2 traffic (~5.4 GB over 15 ms ≈ 370 GB/s) said the kernel was saturating *L2 bandwidth* while DRAM idled at 9 GB/s: a bottleneck one level above the one everyone watches. `const float* __restrict__ in, float* __restrict__ out` is the programmer's signed promise that the pointers don't alias — which the compiler needs because "could a load observe a write?" is a question about *all 16.7M concurrent threads*, not one thread's program order. With the promise, loads compile to `LDG.E.CI` (read-only path through the per-SM cache) — verified by dumping the assembly. The promise costs nothing here: in-place convolution was already a race.

11.3 Interlude — does the bounds check diverge?

Mostly no, and the reason reframes what an `if` costs. A 16×16 block's warp covers 2 rows × 16 columns; only blocks touching an image edge (~1.6% of 65,536 blocks) ever produce lanes that disagree on the bounds test. More fundamentally, the compiler compiled the guarded body with **predication** — compare into a per-lane flag, then `@!P3 LDG`, `@!P3 FFMA`: no branch exists, so nothing can diverge; off-lanes simply discard results. The real cost was counted in the assembly: **~10 instructions per tap, only 1 of them the FMA** — 3–4 of them evaluating a

condition whose answer is "in bounds" for 98.4% of warps. Ask two separate questions of any kernel `if`: do lanes *within a warp* disagree often? and what does evaluating the condition cost when everyone agrees? The second is usually the bill.

11.4 Step 2 — the halo-ring tile (10.6 → 8.4 ms)

Kernel E extends the 1D halo to a ring: a block computing a $TILE_x \times TILE_y$ pixel patch stages $(TILE_x+2r)(TILE_y+2r)$ inputs in shared memory. Two mapping problems are new in 2D:

Loading (flatten-and-stride): 256 threads must fill 640 slots — no 1:1 assignment exists. Number the slots 0..639 row-major; thread t fills slots t, t+256, t+512; each slot unflattens by dividing by the **row width** (slot s → row s/side, col s%side) and maps to a global pixel offset by (−r, −r) from the block's corner. Out-of-image slots get 0.0f — padding baked in, exactly as in 1D. Consecutive slots are consecutive columns: coalesced.

Computing: because the tile starts r up-and-left of the output patch, thread (ty,tx)'s window is simply tile rows ty..ty+2r, cols tx..tx+2r — the window's top-left corner IS (ty,tx), no +r shifts. The classic bug: the tile's row stride is *side* = $TILE_x+2r$, not $TILE_x$, and not W — the wrong stride produces smooth, plausible, wrong-everywhere output, which is what the odd-size (333×479) test cases exist to catch.

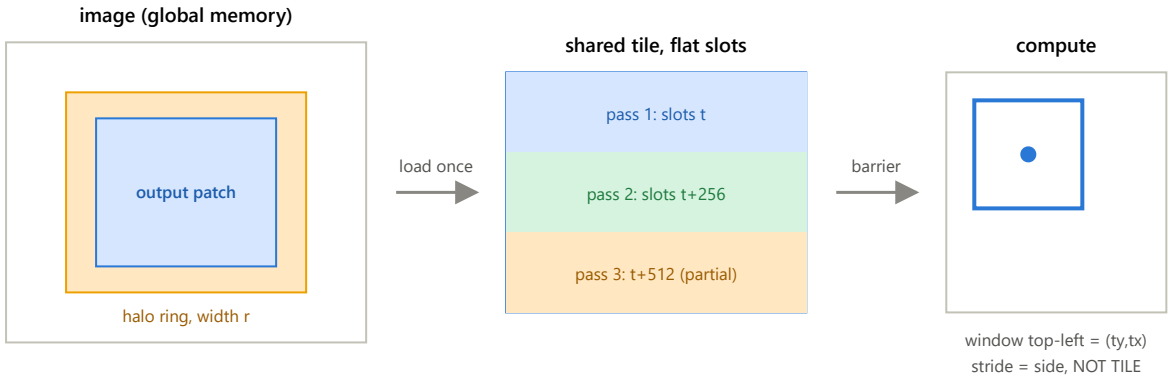


Figure 11.2 — Kernel E. Global reads drop from 81 to 2.25–2.5 per pixel (the tile, incl. halo, over the outputs). Threads whose pixel is outside the image must still load and still reach the barrier — only the final write is guarded. Interactive version: `docs/conv2d_halo_tiling_viz.html`.

11.5 Interlude — warp shape vs the banks

With a 16×16 block, a warp is two rows of 16. During any tap, lanes 0–15 read 16 consecutive tile words at some address A, lanes 16–31 read 16 consecutive words at A+side. With side = 24: banks A..A+15 and (A+24..A+39) mod 32 — lanes 24–31 wrap onto the banks of lanes 0–7. **A 2-way conflict on every shared read.** Padding cannot fix this one — any usable stride still overlaps two 16-wide segments — because the problem is the *shape of the warp*, not the stride. The fix is geometric: **32×8 blocks** put each warp on a single tile row — 32 consecutive words, 32 distinct banks, conflict-free by construction.

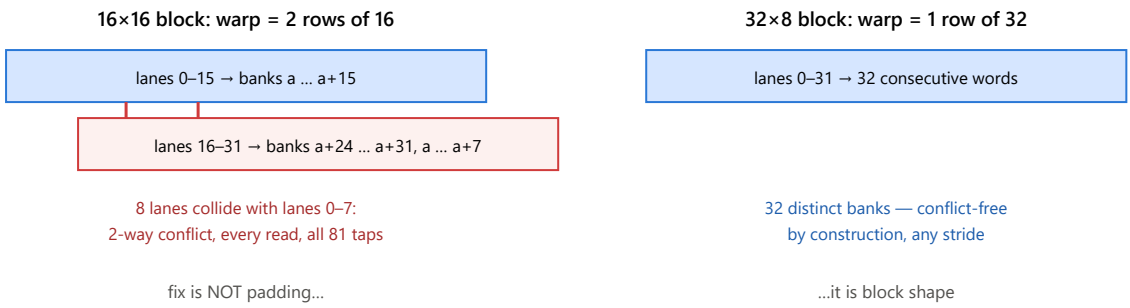


Figure 11.3 — Same 256 threads, different shape. Transpose's conflict came from a strided access pattern (pad fixes it); this one comes from the warp straddling two short rows (reshape fixes it). Diagnose *which kind* you have before reaching for the familiar fix.

The honest twist: making this fix measured **zero** improvement at the 8.4 ms stage — the conflict's cost sat in the shared-memory pipe, *below* the instruction-issue wall that was actually binding. A bottleneck fixed below the waterline doesn't move the boat. It became load-bearing one step later.

11.6 Step 3 — `template <int R>` (8.4 → 3.9 ms)

Reading the compiled generic kernel explained the cap: with runtime r, the compiler had emitted a beautiful 16-taps-at-a-time inner loop — *guarded by "only if >12 taps remain in the row"*. Rows have 9 taps: the fast path was dead code, every row ran remainder loops with per-row address setup, and each tap needed an indexed constant-memory load (`LDC`) for its mask weight (~4–5 instructions and *two* load-pipe operations per tap). With R compile-time and both tap loops unrolled, the mask index becomes a constant per tap and the weight rides *inside* the FMA:

```
FFMA R4, R5, c[0x3][0x4], R4     // mask weight = constant operand: no load instruction
```

Instruction census of the compiled R=4 kernel: **exactly 81 LDS + 81 FFMA** per pixel — two instructions per tap, zero mask loads, zero loop bookkeeping. The remaining time (~3 ms of the 3.9) is the shared-memory pipe itself at one warp-load per cycle per SM — and *this* is where the 32×8 reshape pays: with square blocks that pipe would run at half speed (~6 ms floor).

> **LESSONS**
> (1) Walls fall one at a time, and each is invisible until the previous one falls — so predictions will miss, and the misses are the diagnosis. (2) On sm_61, `__restrict__` is (nearly) free performance whenever overlap would already be a bug. (3) An `if`'s bill is usually condition evaluation, not divergence. (4) Bank conflicts have two species — stride (pad it) and warp-shape (reshape it). (5) An optimization can be correct and premature: measure zero, keep it anyway if the roadmap says the wall it removes is coming. (6) `cuobjdump -sass` + counting instructions is a profiler that can't be permission-blocked. Known headroom: register reuse — one thread computing several neighboring outputs whose windows overlap 8/9 (the matmul register-tiling idea, stencil form).

12 Cross-cutting principles

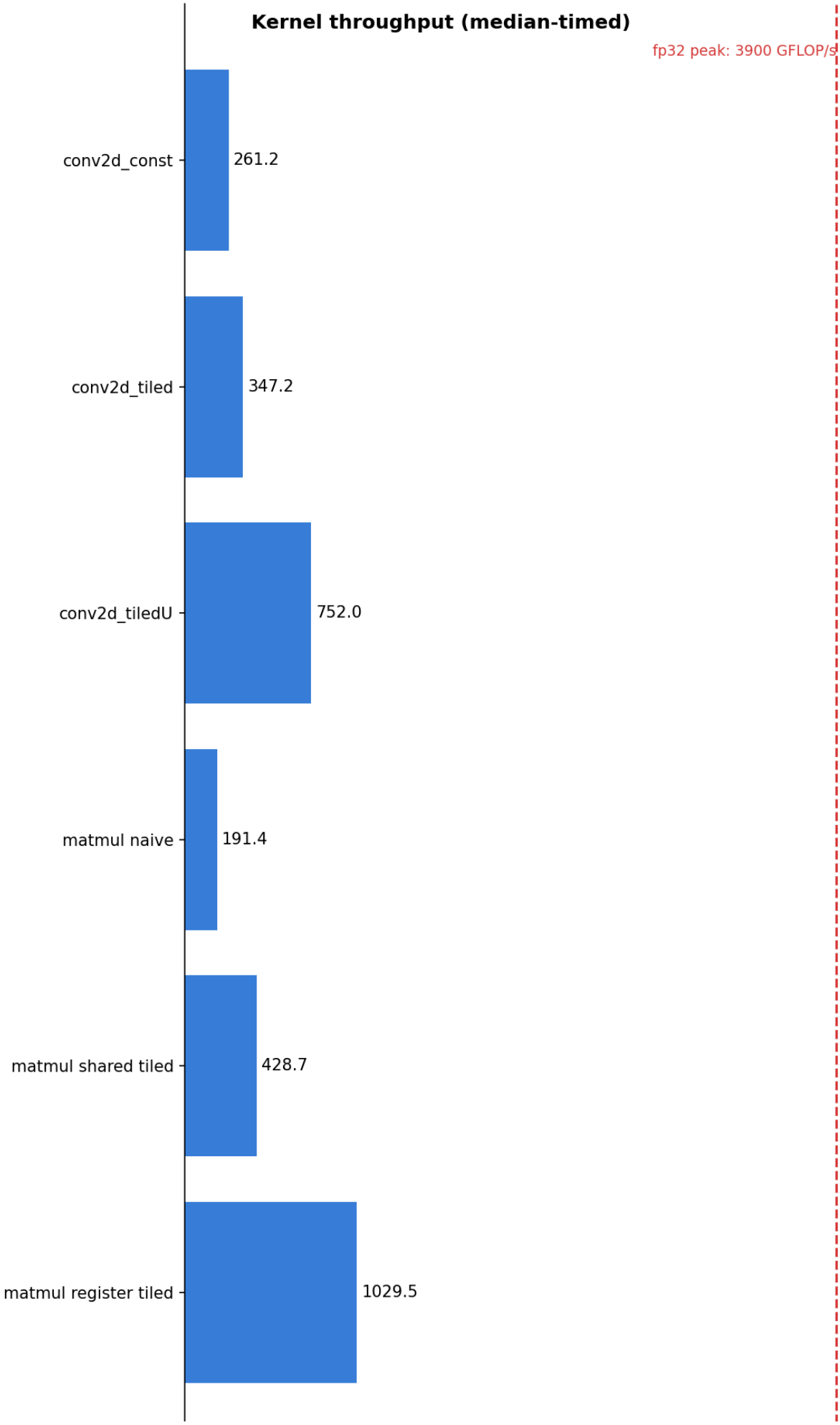
The rules that survived contact with seven kernels. Each cites the chapter where it was paid for.

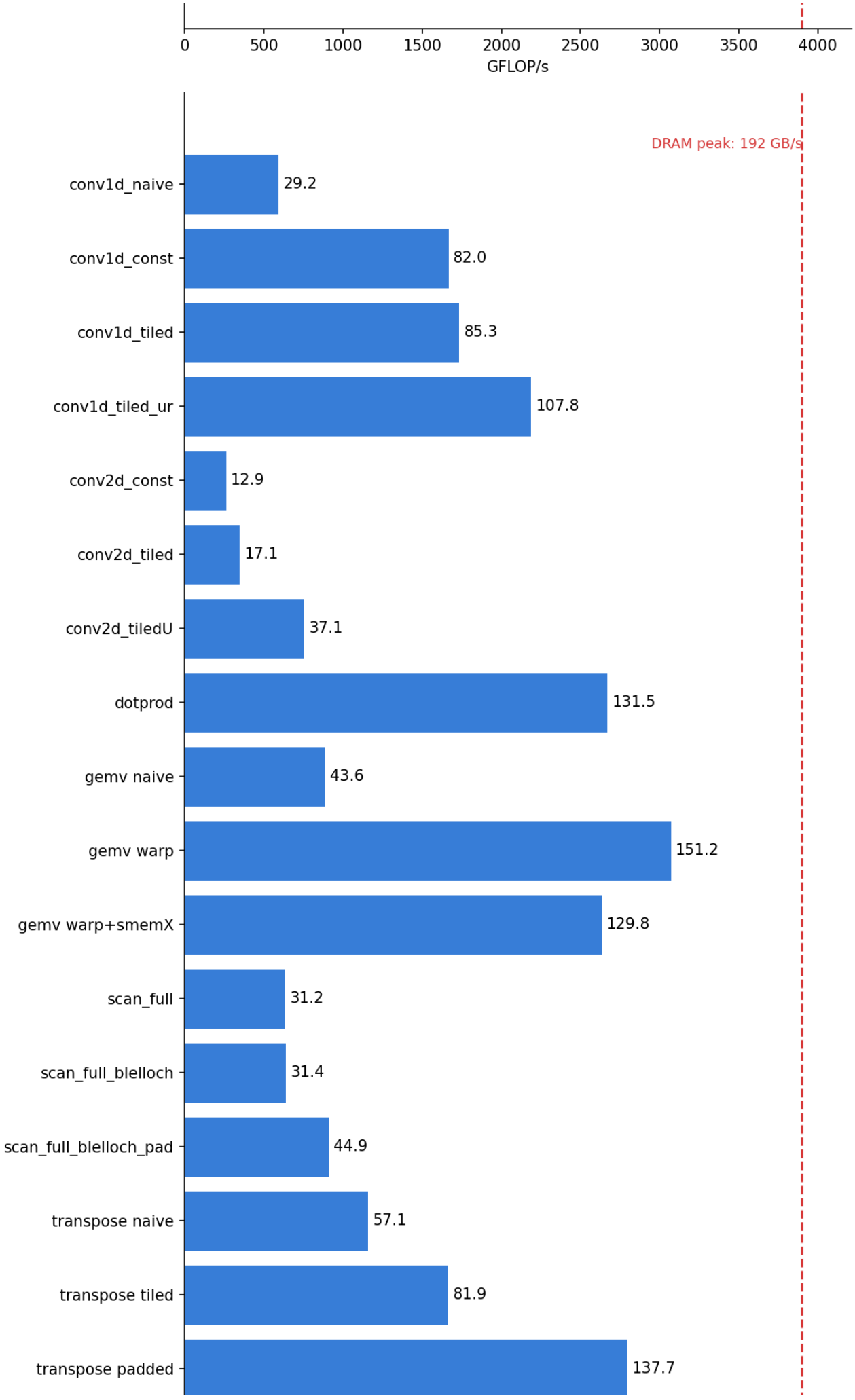
1. **Think in warps, not threads.** Coalescing (3, 8), shuffles (6), divergence and predication (11), bank conflicts (7, 9, 11) — every one is a warp-level effect. The thread is the programming abstraction; the warp is the machine.
2. **Design the thread-to-data mapping first.** The single biggest jumps in the whole project — GEMV 43→151 GB/s, transpose 56→139 — came from changing *who touches what*, not from adding hardware features.
3. **Find the reuse, then choose its home.** Caches (free, unreliable), constant cache (warp-uniform reads only), shared memory (explicit, leased), registers (fastest, per-thread). Matmul climbed exactly this ladder (5); conv climbed it again (10–11).
4. **Shared memory is a purchase.** Price: occupancy + staging + a barrier. Return: reuse × per-access saving vs where the data would otherwise live. GEMV shared-x (8) is the canonical loss; conv tiles are the canonical win; conv1d's r≤2 crossover (10) is the break-even point measured.
5. **Barriers and bank conflicts outrank Big-O.** Work-efficient Blelloch bought 1.07×; removing bank conflicts bought 1.7× (9). Asymptotics count operations; the hardware bills synchronization and serialization.
6. **Bounds checks cost issue slots, not divergence.** Predication means the `if` runs everywhere; ~40% of kernel D's instructions were computing "yes, in bounds" (11). Delete checks structurally (bake padding into data) rather than optimizing them.
7. **Give the compiler compile-time facts.** Runtime parameters block unrolling, constant-operand folding, and strength reduction. `template<int R>` + switch dispatch turned 10 instructions/tap into 2 (10, 11).
8. **Walls fall one at a time.** An optimization below the binding wall measures zero (the 32×8 reshape, 11.5) — that is not evidence it's wrong, only that it's early. Re-measure after each wall falls.
9. **Trust ratios, not absolutes; A/B in the same run.** Laptop clocks drift ±10–50% (1, 9). Correctness first, always: NaN-poisoned outputs, odd sizes, asymmetric masks (4).
10. **Negative results are results.** float4 regressed (6), unrolling was neutral (6), shared-x regressed (8), work-efficiency underwhelmed (9), the reshape measured zero (11). Each is recorded with its reason — collectively they're worth more than the wins.
11. **When the profiler is locked, improvise instruments:** parameter sweeps that decompose time into floor + slope (10); `cuobjdump -sass` instruction censuses (11); designed A/B pairs that isolate one variable (9, 11).

13 Results and glossary

13.1 All measured results

Kernel	Size	Variant ladder	Result
matmul	1024-class	naive → shared-tiled → register-tiled 2×2	~179 → ~430 → ~1044 GFLOP/s
dotprod	16.7M	grid-stride + shuffle/block reduction + atomic	~125–138 GB/s
transpose	2048×4096	naive → tiled → padded [32][33]	~56 → ~82 → ~139 GB/s
GEMV	4096^2	naive → warp-per-row → +shared-x (regression)	~43 → ~151 → ~130 GB/s
scan	1M	3-pass; HS → Blelloch → +padded indexing	pass-1 ratio 1.0 → 1.07 → ~1.7×
conv1d	16.7M, r=4	naive → const → tiled → template-unrolled	~29 → ~82 → ~87 → ~112 GB/s (floor)
conv2d	4096^2, r=4	const → +__restrict__ → tiled 32×8 → template-unrolled	15.0 → 10.6 → 8.4 → 3.9 ms (~750 GFLOP/s)


Kernel throughput (median-timed)
fp32 peak: 3900 GFLOP/s
conv2d_const
261.2
conv2d_tiled
347.2
conv2d_tiledU
752.0
matmul naive
191.4
matmul shared tiled
428.7
matmul register tiled
1029.5


0
500
1000
1500
2000
2500
3000
3500
4000
GFLOP/s
DRAM peak: 192 GB/s
conv1d_naive
29.2
conv1d_const
82.0
conv1d_tiled
85.3
conv1d_tiled_ur
107.8
conv2d_const
12.9
conv2d_tiled
17.1
conv2d_tiledU
37.1
dotprod
131.5
gemv naive
43.6
gemv warp
151.2
gemv warp+smemX
129.8
scan_full
31.2
scan_full_blelloch
31.4
scan_full_blelloch_pad
44.9
transpose naive
57.1
transpose tiled
81.9
transpose padded
137.7

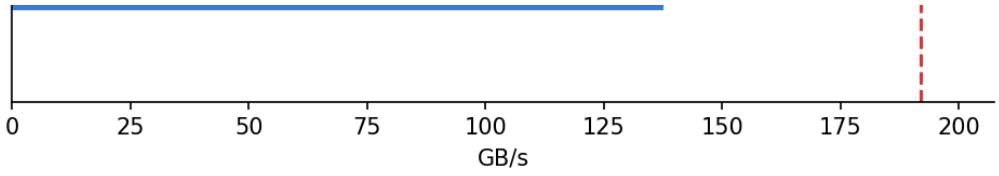


Figure 13.1 — The project scoreboard (`docs/perf.png`), regenerated after every session by `scripts/plot_perf.py`, which runs every test binary and refuses to plot if any correctness check fails.

13.2 Glossary

Term	Meaning
`warp / lane`	32 threads executing one instruction in lockstep / one thread's position (0–31) in it
`coalescing`	hardware merging a warp's 32 loads into few 128-byte transactions; needs consecutive addresses
`occupancy`	resident warps per SM ÷ 64; the SM's supply of "other work" for hiding latency
`bank / bank conflict`	shared memory is 32 banks (word i → bank i mod 32); two lanes, same bank, different words = serialized
`divergence`	lanes of one warp taking different branch paths; each path runs with the others masked off
`predication`	branchless `if`: every lane executes, a per-lane flag (`@!P3`) discards unwanted results
`halo (ghost cells)`	the extra r-wide border of input a tile needs beyond its own outputs, shared with neighbor blocks
`AI / roofline / ridge`	FLOPs per ideal byte / the two-ceiling performance model / the AI where the ceilings meet (~20.3 here)
`effective bandwidth`	ideal (must-move) bytes ÷ time — a score against perfection, not a traffic report
`__constant__`	64 KB read-only region with a per-SM cache that broadcasts warp-uniform reads and folds them into FMA operands
`__restrict__`	programmer's promise that a pointer's memory is reached only through it; unlocks the read-only load path
`SASS / LDS / LDG / LDC / FFMA`	the GPU's real assembly / shared-mem load / global load (.CI = read-only path) / constant-mem load / fused multiply-add
`grid-stride loop`	`for (i = idx; i < N; i += gridDim*blockDim)` — any N, bounded grid, coalesced
`shuffle (__shfl_down_sync)`	register-to-register exchange between lanes of a warp; no memory, no barrier

14 What comes next

- **Histogram** — a new failure mode: thousands of threads *writing* to the same few counters. Technique: privatization (per-block copies in shared memory, merged at the end).
- **Matmul double-buffering** — overlap the load of tile t+1 with computation on tile t; the current 26% of fp32 peak says the FMA units are still waiting on tiles.
- **SpMV (CSR)** — irregular, data-dependent access: load balancing when rows have wildly different lengths.
- **Conv2d register reuse** (optional revisit) — several outputs per thread to cut shared-memory loads per FMA below 1; the current kernel's ~75%-of-time-in-LDS says that's where the next 2× lives.

Interactive companions to these notes (open in any browser): `docs/register_tiled_matmul_viz.html`, `docs/bank_conflict_viz.html`, `docs/memory_hierarchy_occupancy_viz.html`, `docs/blelloch_scan_tree_viz.html`, `docs/conv2d_halo_tiling_viz.html`.